

Matrix Algebra

Part 2/3: Determinants, Inverses and Image Convolution

CSC 1105 Mathematics for Computer Science

Denish Azamuke, PhD

Office: CoCIS, Block B, Level 3

September 7, 2026

Lecture Outline

- Inverse, determinant, and
- Transpose of matrices
- Matrix operations on image processing computations

Learning Outcomes

By the end of this lecture, you should be able to:

- 1 Compute determinants of 2×2 and 3×3 matrices
- 2 Decide whether a square matrix is invertible
- 3 Compute and verify the inverse of a 2×2 matrix
- 4 Solve $A\mathbf{x} = \mathbf{b}$ using inverse and elimination methods
- 5 Extract grayscale and RGB matrices from an image
- 6 Apply a convolution kernel and express it as matrix multiplication

Determinant of a 2×2 Matrix

For

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad \det(A) = ad - bc.$$

Example

$$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$$

$$\det(A) = 4(6) - 7(2) = 24 - 14 = \boxed{10}.$$

Since $\det(A) \neq 0$, the matrix is invertible.

Determinant Rules

For square matrices of the same order:

$$\det(AB) = \det(A) \det(B),$$

$$\det(A^T) = \det(A),$$

$$\det(A^{-1}) = \frac{1}{\det(A)} \quad \text{when } A^{-1} \text{ exists.}$$

A 3×3 Determinant by Cofactor Expansion

For a matrix $A = [a_{ij}]$, the determinant can be expanded along row i using

$$\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{ij} M_{ij}$$

where M_{ij} is the determinant obtained by deleting row i and column j .

For a 3×3 matrix expanded along the first row,

$$\det(A) = a_{11}M_{11} - a_{12}M_{12} + a_{13}M_{13}.$$

Let

$$B = \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 5 & 2 & 0 \end{bmatrix}.$$

Expanding along the first row:

$$\begin{aligned} \det(B) &= 2 \begin{vmatrix} -1 & 4 \\ 2 & 0 \end{vmatrix} - 1 \begin{vmatrix} 0 & 4 \\ 5 & 0 \end{vmatrix} + 3 \begin{vmatrix} 0 & -1 \\ 5 & 2 \end{vmatrix} \\ &= 2(-8) - 1(-20) + 3(5) = \boxed{19}. \end{aligned}$$

Note: The alternating signs across the first row are $+, -, +$.

Determinant by Elimination

Row replacement preserves the determinant:

$$\begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 5 & 2 & 0 \end{bmatrix} \xrightarrow{R_3 \leftarrow R_3 - \frac{5}{2}R_1} \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 0 & -\frac{1}{2} & -\frac{15}{2} \end{bmatrix}$$

$$\xrightarrow{R_3 \leftarrow R_3 - \frac{1}{2}R_2} \begin{bmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 0 & 0 & -\frac{19}{2} \end{bmatrix}.$$

For an upper triangular matrix, multiply the diagonal entries:

$$\det(B) = 2(-1) \left(-\frac{19}{2} \right) = \boxed{19}.$$

Elimination scales better than cofactor expansion for large matrices.

Inverse of a Matrix

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ with $ad - bc \neq 0$,

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

For $A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$, we already found $\det(A) = 10$:

$$A^{-1} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}.$$

Verification

$$AA^{-1} = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2.$$

Solving $A\mathbf{x} = \mathbf{b}$

Let

$$\begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Because A is invertible,

$$\mathbf{x} = A^{-1}\mathbf{b} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \boxed{\begin{bmatrix} 0.6 \\ -0.2 \end{bmatrix}}.$$

Check:

$$\begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} 0.6 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 2.4 - 1.4 \\ 1.2 - 1.2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Images are Arrays of Numbers



The sample image becomes an RGB array:

$$X \in \{0, \dots, 255\}^{360 \times 270 \times 3}.$$

At each pixel (i, j) :

$$X_{ij} = \begin{bmatrix} R_{ij} & G_{ij} & B_{ij} \end{bmatrix}.$$

The first stored pixel is

$$\begin{bmatrix} 66 & 53 & 53 \end{bmatrix}.$$

RGB to Grayscale as a Linear Combination

The three color channels combine into one grayscale matrix:

$$Y = 0.299R + 0.587G + 0.114B.$$

Thus,

$$Y \in \mathbb{R}^{360 \times 270},$$

where each entry represents the grayscale intensity of one pixel.

In NumPy, if

$$X.\text{shape} = (360, 270, 3),$$

then

$$Y = X @ \text{np.array}([0.299, 0.587, 0.114]).$$



A Pixel Patch is a Matrix



A 6×8 patch from the grayscale image:

212	222	228	226	219	221	226	226
201	203	208	196	212	225	227	224
178	177	164	168	199	224	224	226
164	123	159	173	178	190	194	204
135	141	153	161	157	149	167	184
156	174	155	155	139	148	148	155

Rows record vertical position; columns record horizontal position.

Transpose of an Image Matrix

Original grayscale matrix



$$Y : 360 \times 270$$

The entry relation is $(Y^T)_{ij} = Y_{ji}$.

Transposed matrix



$$Y^T : 270 \times 360$$

Convolution as a Local Matrix Computation

Let P_{ij} be the 3×3 image patch centred at output position (i, j) , and let K be a 3×3 kernel.

$$y_{ij} = \langle P_{ij}, K \rangle_F = \sum_{u=1}^3 \sum_{v=1}^3 P_{ij}(u, v) K(u, v).$$

Interpretation

Multiply corresponding entries, then add the nine products. Slide the kernel across the image and repeat.

Many software libraries implement cross-correlation without flipping K but use the name convolution.

3 × 3 Kernels

Box blur

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Smooths local variation.

Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Emphasizes local contrast.

Sobel horizontal change

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Responds to vertical edges.

The blur entries sum to 1. Edge-detection entries sum to 0.

Blur and Sharpen on the Banknote Matrix

Box blur

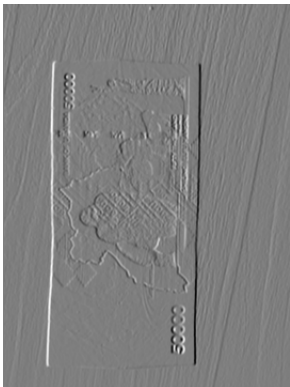


Sharpen

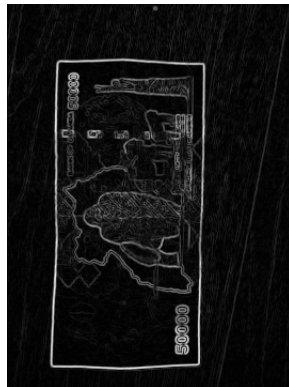


Sobel Response and Edge Magnitude

Signed x -direction response



Combined magnitude



Sobel filters applied to a banknote image

Convolution as Matrix Multiplication

A 3×3 kernel contains 9 entries. Flatten it into a vector:

$$\mathbf{k} = [k_{11} \ k_{12} \ k_{13} \ k_{21} \ k_{22} \ k_{23} \ k_{31} \ k_{32} \ k_{33}]^T \in \mathbb{R}^9.$$

Similarly, flatten each 3×3 image patch into a vector $\mathbf{p}_i \in \mathbb{R}^9$ and stack the patches as rows:

$$P = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \vdots \\ \mathbf{p}_N^T \end{bmatrix}.$$

Each filter output is the dot product of a patch and the kernel, so all outputs can be computed at once as

$$\boxed{\mathbf{y} = P\mathbf{k}}.$$

Key Idea

Convolution can therefore be reorganized as a standard matrix–vector multiplication.

Revision Questions

- 1 Compute the determinant and inverse of

$$C = \begin{bmatrix} 3 & 5 \\ 1 & 2 \end{bmatrix}.$$

- 2 Verify $CC^{-1} = I_2$.
- 3 Choose another '.jpg' image from a dataset and extract a 5×5 grayscale patch.
- 4 Apply the blur and Sobel kernels. Explain the numerical meaning of one output entry.
- 5 Predict the size of the image-to-columns matrix for a 10×10 image and a valid 3×3 filter.

Questions?